MODULE NAME:	MODULE CODE:
PROGRAMMING 3A	PROG7311
ENTERPRISE APPLICATION DEVELOPMENT	EAPD7111/w

ASSESSMENT TYPE: POE (PAPER & MARKING RUBRIC)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A minimum of 45 HOURS is suggested to complete this assessment

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. ***Please ensure that you submit your assignment through Turnitin.***
3. ***Make a copy of your assignment before handing it in.***
4. *Assignments must be typed unless otherwise specified.*
5. *Begin each section on a new page.*
6. *Follow all instructions on the PoE cover sheet.*
7. *This is an individual assignment.*

© The Independent Institute of Education (Pty) Ltd 2026

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate.** The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage.**
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage.**
- **Where both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: **Consistent and congruent referencing**	**Minor errors** **Deduct 5% from overall percentage.** **Example: if the response receives 70%, deduct 5%. The final mark is 65%.**	**Major errors** **Deduct 10% from the overall percentage.** **Example: if the response receives 70%, deduct 10%. The final mark is 60%.**
Consistency • The **correct referencing style** for the **discipline** – i.e., either **Harvard**, OR **APA** (for Psychology), OR **Law**, OR **IEEE** (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. • **Concepts** and **ideas** that are **quoted** and/or **paraphrased** are referenced consistently throughout. • **Position of the in-text reference**: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	**Minor inconsistencies:** • The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. • For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. • Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. • Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	**Major inconsistencies:** • Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. • Multiple referencing styles for the same source types have been used. • For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. • Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. • Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency • **Each source** reflected within **in-text references** is included accurately in the **bibliography/reference list.** • All **bibliography/reference list** entries are in the **required order** for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). • All **direct quotes** and **paraphrases** have been **integrated appropriately** into the text using introductory phrases, accurate grammar, etc.	**Minor incongruences:** • There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). • There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. • An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	**Major incongruences:** • No relationship/several incongruencies between the in-text referencing and the bibliography/reference list. • For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. • Most sources are listed in a haphazard order throughout the bibliography/reference list. • Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

© The Independent Institute of Education (Pty) Ltd 2026

Global Logistics Management System (GLMS)

Background: "TechMove Logistics" is a global shipping coordinator that manages international freight contracts, driver schedules, and service invoices. Currently, they rely on a legacy system comprising disjointed spreadsheets, emails, and manual phone calls. This outdated approach has resulted in severe data fragmentation, lost invoices, compliance failures regarding expired contracts, and significant workflow bottlenecks.

TechMove has commissioned your software development firm to build the **Global Logistics Management System (GLMS)**. This must be an **Enterprise-Grade** solution capable of handling complex relationships between clients, contracts, and service requests while ensuring high availability and scalability.

Objective: The primary goal of this PoE is to develop a scalable, distributed enterprise web platform. You will architect the solution using industry standards, build a robust prototype with automated testing, and finally deploy a modernised, containerised version using Docker.

Key Features of the Platform:

1. **Contract Management Hub:**
   - A centralised repository for storing complex legal agreements, client details, and Service Level Agreements (SLAs).
   - Automated status tracking (Draft, Active, Expired, On Hold).
2. **Service Request Processing:**
   - Logistics managers can raise service requests linked to specific contracts.
   - The system must validate that requests are only raised against valid, active contracts.
3. **Financial Integration:**
   - The system must handle international currency conversions automatically using external data sources (APIs).
4. **Modern Architecture:**
   - The solution must transition from a monolithic architecture to a service-oriented, containerised environment to support future scaling.

© The Independent Institute of Education (Pty) Ltd 2026

Instructions

Complete the parts below to provide the solution for TechMove Logistics.

Tip: This project evolves. What you design in Part 1, you build in Part 2, and you "productionize" (Docker & API) in Part 3.

Note on Testing: In Parts 2 and 3, you are required to implement automated testing. Enterprise systems cannot rely on manual testing alone; you must prove your logic is sound through code.

© The Independent Institute of Education (Pty) Ltd 2026

Part 1 — Enterprise Analysis & Architecture Report (Marks: 100)

Learning Units covered:

LU1: *Characteristics of Enterprise Software Systems (Scalability, Availability).*

LU2: *Design and Architecture Patterns and Architecture Frameworks (TOGAF/Zachman).*

Scenario: You are the **Chief Solutions Architect**. You need to sell the technical vision to the TechMove Board of Directors before a single line of code is written. The board is concerned about how this new software will integrate with their current operations and handle future growth.

1. Architectural Framework Strategy:

- Select **one** Enterprise Architecture Framework (**Zachman** or **TOGAF**) to structure the project.
- Create a high-level visual representation of the GLMS using your chosen framework.
  - **If Zachman:** Map out the Data, Function, and Network columns against the Planner and Designer rows specifically for the logistics context.
  - **If TOGAF:** Briefly explain the ADM Cycle application for this specific project (Phases A through D).

2. Design Pattern Selection: Identify **three** specific GoF (Gang of Four) design patterns you will implement in the code. *Note: Simple patterns like Singleton alone are insufficient.*

- **Requirement:** Provide a **UML Class Diagram** showing exactly how these patterns will be structured in your future C# code.

3. Non-Functional Requirements & Scalability:

- Define the **Availability** and **Interoperability** requirements for the system.
- **Scalability Strategy:** If TechMove acquires a competitor and adds 50,000 new contracts overnight, how will your architecture handle the load? Discuss specific strategies such as **Horizontal Scaling**, **Load Balancing**, or **Caching**.

Submission Guidelines:

- Format your report as a PDF
- Length: 1000 – 1500 words (excluding diagrams).
- Include all diagrams within the document.

© The Independent Institute of Education (Pty) Ltd 2026

Part 2 — The Core Prototype & Unit Testing (Marks: 100)

Learning Units covered:

- ***LU3:*** *Enterprise Software System Development.*
- ***LU4:*** *Optimising Application Performance (Async/Await).*
- ***Quality Assurance:*** *Test-Driven Development (Unit Testing).*

Task: Develop the initial functional prototype using **ASP.NET Core MVC**. This version will be a "Monolith" to establish core functionality and business logic.

Building a "Monolith" to establish core functionality and business logic is a standard, often recommended approach for initial software development, particularly for Minimum Viable Products (MVPs) or startups. This approach involves constructing the application as a single, unified unit where the user interface, database access, and business logic are tightly coupled.

1. Database & Complex Models:

- Implement a SQL Server database using **Entity Framework Core** .
- **Entities Required:**
  - Client: (Name, Contact Details, Region).
  - Contract: (Linked to Client, StartDate, EndDate, Status [Draft/Active/Expired], ServiceLevel).
  - ServiceRequest: (Linked to Contract, Description, Cost, Status).
- **File Handling:** Users must be able to upload a PDF "Signed Agreement" for every Contract. This file must be saved to the server (simulate a file server) and be downloadable via the UI.

2. Workflow Logic:

- Implement the status workflow. A ServiceRequest cannot be created if the parent Contract is set to "Expired" or "On Hold."
- Implement a **Search/Filter** mechanism using LINQ to allow Admins to find contracts by Date Range and Status.

© The Independent Institute of Education (Pty) Ltd 2026

3. External API Integration (Consumption):

- TechMove deals with international currencies (USD, EUR, etc.) but reports in ZAR.
- **Requirement:** On the ServiceRequest creation page, you must consume a free external **Currency Exchange API** (e.g., *ExchangeRate-API* or *OpenExchangeRates*).
- Use HttpClient to get the current USD-to-ZAR rate. When the user types a dollar amount, the system must auto-calculate and save the local cost.

4. Unit Testing (Quality Assurance):

- **Requirement:** You must create a separate **Test Project** in your solution using **xUnit** or **NUnit**.
- **What to Test:** You must write unit tests for your Business Logic, specifically:
  - **Currency Calculation:** Verify that the math converting USD to ZAR is correct, given a specific rate.
  - **File Validation:** Verify that uploading a restricted file type (e.g., .exe) throws an error (only .pdf allowed).
  - **And any big system tasks that can be tested**
- **Evidence:** Provide screenshots showing all tests passing in the Test Explorer.

Submission Guidelines:

- Visual Studio Solution on GitHub and link submitted on ARC. No GitHub submission will result in 5% removed from your assignment
- Database Migration Scripts.
- Test Execution Screenshots.
- A full, detailed video submission showing your application running and a detailed explanation of the flow of your application. This is key, as this is what your lecturer will use to mark your assignment as well.

© The Independent Institute of Education (Pty) Ltd 2026

Part 3 — Modernisation, Docker & Automated Testing (Marks: 100)

Learning Units covered:

- *All Learning Units.*
- *Quality Assurance: Automated Integration Testing.*

Task: The prototype (Part 2) was a success. Now, you must decouple the system into a **Service-Oriented Architecture (SOA)**, containerise it for "Cloud-Native" deployment, and implement automated integration tests.

1. Refactoring: The Web API (The Backend):

- Strip the business logic and database access out of your MVC Controllers.
- Create a **New Project** in your solution: An **ASP.NET Core Web API**.
- The API should be the system talking to the DB , important !
- This API must expose endpoints for:
  - GET /api/contracts (with filtering).
  - POST /api/contracts (to create new ones).
  - PATCH /api/contracts/{id}/status (to approve/decline).
  - And any other you might need to make your system work correctly. These are just examples
- **Swagger/OpenAPI:** Enable Swagger so the API is self-documenting and testable.
- Reuse the code within your API and allow the API to return JSON data

2. Refactoring: The MVC Client (The Frontend):

- Modify your existing MVC Project. It should **no longer connect to the SQL Database directly**.
- Instead, the MVC Controllers must use HttpClient to call your new **Web API** to get and save data.
- *Why?* This separates the "Presentation Layer" from the "Service Layer," allowing them to scale independently.
- Make sure there is some authentication included(JWT, Firebase or others)

3. Automated API Integration Testing:

- Expand your Tests project to include **Integration Tests**

© The Independent Institute of Education (Pty) Ltd 2026

- **Task:** Write code that automatically calls your *running* API endpoints and asserts the response.
    - *Example:* Call GET /api/contracts, assert that the HTTP Status Code is 200 OK, and verify the returned JSON is not null.
- *Why?* In a DevOps pipeline, these tests run automatically to prevent "breaking changes" before deployment.

4. Full Containerization (Docker Compose):

- You must create a **docker-compose.yml** file that spins up the entire ecosystem.
    - **Container 1:** sql-server-db (The Database).
    - **Container 2:** glms-backend-api (Your Web API - connects to Container 1).
    - **Container 3:** glms-frontend-web (Your MVC App - connects to Container 2).
- **Networking:** Ensure the containers can talk to each other using internal Docker networking.

5. Technical Reflection Report:

- **DevOps & Testing:** Explain why **Automated Testing** is critical in a CI/CD pipeline. How does it prevent bugs from reaching production?
- **Containerization:** Discuss how Docker ensures consistency across Dev, Test, and Prod environments, solving the "it works on my machine" problem.

Submission Guidelines:

- Full Source Code (Solution with two projects: MVC and API) all inside GitHub.
- Dockerfile for the API & Dockerfile for the Web App, all inside GitHub.
- docker-compose.yml all inside GitHub.
- No GitHub submission will result in 5% removed from your assignment.
- Screenshots of the system running via Docker Desktop.
- PDF of your technical Reflection Report.
- A full, detailed video submission showing your application running and a detailed explanation of the flow of your application. This is key, as this is what your lecturer will use to mark your assignment as well.

© The Independent Institute of Education (Pty) Ltd 2026

Assessment Sheet (Marking Rubric)

Please note: Tear off this section and **attach** it to your work when you submit it/ If this is an online submission, then this information needs to be included in the online submission.

MODULE NAME:	MODULE CODE:
PROGRAMMING 3A	PROG7311
ENTERPRISE APPLICATION DEVELOPMENT	EAPD7111/w

STUDENT NAME:
STUDENT NUMBER:

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
1. Executive Summary & Introduction Contextualization of the TechMove project. **[5 Marks]**	• No summary provided. • Context is missing.	• Brief summary of the project goals. • Context is generic.	• Good summary linking the technical solution to business problems (Legacy systems).	• Professional executive summary that clearly defines the scope, audience, and architectural vision.	
	0 Marks	**1-2 Mark**	**3-4 Marks**	**5 Marks**	
2. Enterprise Framework Strategy (Theory) Understanding of Zachman or TOGAF. **[10 Marks]**	• Framework selection is missing or incorrect. • Explanation is factually incorrect.	• Framework selected. • Definitions are standard/textbook-based.	• Good explanation of the framework's phases or columns.	• Deep theoretical understanding shown. • Explains *why* the specific framework was chosen for a Logistics company.	
	0 – 2 Mark	**3-5 Marks**	**6-7 Marks**	**8-10 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
3. Framework Application (Practical) Applying the framework to TechMove Data/Process. **[15 Marks]**	• Application is generic (copy-paste). • No visual representation.	• Basic attempt to map framework to scenario (e.g. mentions "Data"). • Visual representation is messy.	• Good mapping. Specific logistics entities (Contracts, Drivers) appear in the framework matrix/phases.	• Expert application. The visual diagram is custom-made and maps the specific TechMove data flows to the framework pillars perfectly.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
4. Design Patterns: Selection & Justification **[15 Marks]**	• Patterns missing or inappropriate (e.g. Singleton only) • No justification.	• 3 Patterns identified. • Justification is weak ("It's good for code").	• Good justification linking Factory to "Contract Types" and Observer to "Notifications".	• Excellent justification. Explains how the patterns solve the decoupling and validation problems specific to TechMove.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
5. Design Patterns: Logic & Structure Technical description of implementation. **[15 Marks]**	• Logic description is missing. • Incorrect understanding of how the pattern works.	• Describes what the pattern does generically.	• Describes the specific Classes and Interfaces involved (e.g., IContractFactory).	• Detailed technical breakdown of the relationships, dependency injection strategies, and event triggers.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
6. UML Modelling Technical diagramming skills. **[20 Marks]**	• UML missing. • Not a Class diagram. • Syntax is completely wrong.	• Diagram exists but has syntax errors (wrong arrows). • Mismatch with text.	• Diagram is technically accurate. • Correct notation for Inheritance/Composition.	• Industry-standard UML. • Perfectly reflects the 3 patterns. • Includes visibility (+/-), data types, and cardinality.	
	0 – 4 Marks	**5-10 Marks**	**11-14 Marks**	**15-20 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
7. Scalability & Non-Functional Requirement Strategy Load Balancing, Caching, Availability. **[20 Marks]**	• Non-Functional Requirement missing. • Scalability not addressed.	• Definitions of Availability/Scalability provided • Strategy is vague ("Add more servers").	• Good distinction between Horizontal vs Vertical scaling. • Mentions specific tools (Load Balancers).	• Comprehensive strategy • Proposes specific Caching layers (Redis) and Load Balancing techniques to handle the "50,000 contracts" surge.	
	0 – 4 Marks	**5-10 Marks**	**11-14 Marks**	**15-20 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
1. Database Architecture (EF Core) Schema, Keys, Connections. **[10 Marks]**	• Database crashes or is missing. • Hardcoded connection strings.	• EF Core configured • Tables exist but lack relationships.	• Schema is correct (Client, Contract, Request). • One-to-Many relationships work.	• Perfectly normalised schema. • Fluent API or Attributes used for constraints. • Connection string in appsettings.json.	
	0 – 2 Marks	**3-5 Marks**	**6-7 Marks**	**8-10 Marks**	
2. Design Pattern Implementation (From Part 1) Translating UML to Code. **[20 Marks]**	• No patterns implemented. • Controllers call DbContext directly • Logic is tightly coupled.	• Very basic implementation of design patterns used	• Moderate implementation of design pattern	• Complete implementation of design patterns used.	
	0 – 4 Marks	**5-10 Marks**	**11-14 Marks**	**15-20 Marks**	
3. Workflow & Validation Logic Business Rules (Status checks). **[15 Marks]**	• No validation. • "Expired" contracts allow requests.	• Basic validation (Required fields). • Workflow logic has bugs.	• Correctly prevents requests on Expired contracts. • Status transitions work.	• Robust validation logic (Server-side & Client-side). • Edge cases handled gracefully. • Validation logic is separated from Controllers.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
4. File Handling Mechanism PDF Uploads. **[10 Marks]**	• Feature missing. • Files not saved.	• File upload works but saves to DB (blob) instead of disk. • No extension validation.	• Files save to server folder. • Path stored in DB. • Basic validation.	• Robust handling (UUID naming to prevent overwrites). • Strict validation (PDF only). • Download link works.	
	0 – 2 Marks	**3-5 Marks**	**6-7 Marks**	**8-10 Marks**	
5. External API Integration Currency conversion logic. **[15 Marks]**	• No API integration. • Code blocks thread (.Result).	• Connects to API. • Logic is buggy or calculation is incorrect.	• HttpClient used correctly with Async/Await. • Data applied to user input correctly.	• Flawless integration. • Includes error handling (try/catch) if API is down. • Math is precise.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
6. Unit Testing: Setup & Coverage Test Project configuration**.** **[15 Marks]**	• No Test project. • Tests fail to build.	• No Test project. • Tests fail to build.	• Tests cover Currency Math AND File Validation. • Most tests pass.	• Comprehensive test suite. • Mocks used if necessary. • All tests pass (Green). • Automated in GitHub Action	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
7. Unit Testing: Quality Depth of testing. **[15 Marks]**	• Tests irrelevant.	• Happy path testing only (Expected input).	• Tests check for some failures (Invalid input).	• Tests cover edge cases (0 value, .exe file, null inputs). • Demonstrates TDD principles.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
No GitHub used **[-5%]**	No GitHub used or submitted.				
	-5%				

© The Independent Institute of Education (Pty) Ltd 2026

PART 3					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
1. Web API Construction REST Endpoints & Verbs. **[15 Marks]**	• API Project missing. • Endpoints return HTML.	• API exists. • Wrong verbs used.	• Correct REST architecture. • GET, POST, PATCH work correctly.	• Professional API structure. • Correct HTTP Status codes (201, 404, 400). • Route attributes used correctly.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
2. Architectural Integrity (Patterns) Repository & Separation of Concerns in API. **[15 Marks]**	• Patterns from Part 2 were deleted. • API Controllers contain SQL logic directly.	• **Repository:** Used in API, but the structure is messy. • Logic leaks into Controllers.	• **Repository:** API uses Repositories correctly. • **Service Layer:** Logic is separated from API endpoints.	• Excellent Architecture. • Dependency Injection used extensively. • Patterns successfully moved to the Backend logic layer without breaking.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
3. Frontend Refactoring (Decoupling) Removing SQL dependencies. **[15 Marks]**	• MVC still connects to SQL. • No refactoring.	• Hybrid approach (some API, some SQL).	• MVC is fully decoupled. • Controllers use HttpClient only.	• Clean "Service Layer" implementation in Frontend. • Zero database dependency. • Error handling for failed API calls.	
	0 – 4 Marks	**5-7 Marks**	**8-11 Marks**	**12-15 Marks**	
4. Automated Integration Testing Testing running endpoints. **[20 Marks]**	• No integration tests. • Tests do not run.	• Tests exist but are brittle/fail. • Only checks "Not Null".	• Tests successfully call the API. • Asserts SuccessStatusCode (200).	• Robust Integration suite. • Verifies data integrity (Create then Read). • Evidence of passing tests provided.	
	0 – 4 Marks	**5-10 Marks**	**11-14 Marks**	**15-20 Marks**	

© The Independent Institute of Education (Pty) Ltd 2026

PART 3					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
5. Docker Configuration (Files) Dockerfiles Structure. **[10 Marks]**	• No Dockerfiles. • Generated files untouched/broken.	• Dockerfiles exist. • Basic configuration.	• Correct Multi-stage builds used. • Port exposition correct.	• Optimised Dockerfiles. • Efficient layering. • Entrypoints configured correctly.	
	0 – 2 Marks	**3-5 Marks**	**6-7 Marks**	**8-10 Marks**	
6. Container Orchestration Docker Compose & Networking. **[20 Marks]**	• No Compose file. • System cannot run.	• Compose file exists. • Networking broken (Containers can't talk).	• Builds successfully. • DB, API, Web containers run and communicate.	• Flawless orchestration. • Networking uses service names (glms-api). • Environment variables passed via Compose.	
	0 – 4 Marks	**5-10 Marks**	**11-14 Marks**	**15-20 Marks**	
7. Technical Reflection **DevOps & CI/CD insight.** **[5 Marks**	• Report missing.	• Generic definitions of DevOps.	• Good explanation of Container consistency.	• Insightful link between Automated Testing and CI/CD. • Clear explanation of "Works on my machine" solution.	
	1 Mark	**2 Marks**	**3-4 Marks**	**5 Marks**	
No GitHub used **[-5%]**	No GitHub used or submitted.				
	-5%				

[TOTAL MARKS: 300]

© The Independent Institute of Education (Pty) Ltd 2026